

coupledFoam explained

A tutorial companion to the paper “coupledFoam: a block-coupled pressure–velocity solver with single-precision block-multigrid linear algebra for steady incompressible RANS in OpenFOAM”

The coupledFoam project

Generated at commit `bdf`e108

Abstract

This is the plain-language version of the coupledFoam paper. It is written for an engineer who runs OpenFOAM simulations and wants to understand what the new solver does differently from `simpleFoam`, why each of its parts is there, and how to read its results. Every concept is introduced from intuition first; equations appear only where they make an idea sharper. The document uses exactly the same measured numbers, tables and figures as the professional paper: they are generated automatically from the result files, and anything that has not been measured yet appears as a boxed “pending” placeholder rather than as a guess.

Contents

How to use this document	2
1 What problem are we solving?	3
1.1 The flow equations in words	3
1.2 Why pressure is the awkward one	4
1.3 From equations to numbers: finite volumes	4
1.4 Residuals: how far are we from the answer?	5
2 The standard approach: SIMPLE and simpleFoam	5
2.1 Guess and correct	5
2.2 Why SIMPLE needs under-relaxation	6
2.3 Why it takes so many iterations	6
3 The coupled idea	7
3.1 Solve everything together	7
3.2 The 4×4 block structure	7
3.3 Outer and inner iterations	7
3.4 The trade-off, and the only fair comparison	8
3.5 A map of one outer iteration	8
4 The building blocks	9
4.1 Boundary conditions and parallel interfaces	9
4.2 Convection schemes and deferred correction	9

4.3	Rhie–Chow interpolation and the checkerboard problem	10
4.4	Pseudo-transient continuation and the local CFL number	11
4.5	Line search and physicality checks	13
4.6	Remediation: special treatment for problem cells	13
4.7	Sentinel and rollback	14
4.8	Restart	14
4.9	Krylov solvers: GMRES, FGMRES and BiCGStab	15
4.10	Preconditioning	15
4.11	Algebraic multigrid	16
4.12	Smoothers: why Gauss–Seidel failed and ILU(0) worked	18
4.13	Eisenstat–Walker: do not over-solve	18
4.14	Anderson acceleration (optional)	19
4.15	Processor agglomeration: gathering coarse levels	19
4.16	Float versus double: the precision design	19
4.17	Fixing the pressure level in closed domains	21
4.18	Turbulence, MRF and the rest	21
5	The test cases	21
5.1	Primer: how to read the results	21
5.2	Test protocol	23
5.3	T0: lid-driven cavity	23
5.4	T1–T5: from 2D turbulence to 3D vehicles	26
6	Performance: is it faster?	27
6.1	The benchmark protocol in plain words	27
6.2	How to read speed-ups	28
6.3	The benchmark figures	29
7	What this means for the F1 half-car	30
8	Limitations	32
9	Summary: the ideas in ten sentences	33
	Glossary	33

How to use this document

The professional paper (`paper.pdf`) is written for CFD numerics specialists. This tutorial follows the same path in more steps. You can read it front to back, or jump to the building block you are curious about: each one in Section 4 is self-contained.

Four kinds of boxes appear throughout:

Intuition: the blue box

An everyday picture of an idea. It is deliberately imprecise; the text next to it says what is actually done.

Why it matters: the green box

The practical consequence for you as a user: what you will see in a run, which setting it affects, or which result it explains.

Pitfall: the orange box

Something that went wrong during development or that easily goes wrong in practice, and what was done about it.

Building block: the grey box

What is it? A short definition.

Why do we need it? Its job inside coupledFoam.

What would go wrong without it? The failure you would see if it were missing.

Numbers. All measured values in this document are inserted automatically from the file `numbers.tex`, which the report generator `bench/make_report.py` writes from the JSON result records in `results/`. A small box `pending` marks a number that has not been measured yet; a large box marked *Results pending* stands for a whole figure or table. Numbers that are *settings* (for example the default maximum CFL number) or *acceptance thresholds* (for example “within 1 %”) are part of the design, not measurements, and are written directly.

Cross references. Where a statement is documented in detail in the professional paper or in the project’s decision log `DECISIONS.md`, the item is named in the text, e.g. “(D-028)”, and a small superscript such as [P: *Residual norms*] names the section of the professional paper where the full details are. The decision log records every point where the implementation deviates from, or had to interpret, the original specification.

1 What problem are we solving?

1.1 The flow equations in words

For a steady, incompressible flow of air around a car two physical laws have to hold in every small piece of the fluid:

1. **Momentum balance (Newton’s second law).** The change of momentum of a fluid parcel equals the forces acting on it. The forces are the pressure pushing on it from all sides and the friction (viscous and turbulent stresses) exerted by the neighbouring fluid. In a steady flow the “change of momentum” is purely the momentum carried in and out by the flow itself, which is called *convection*.
2. **Mass balance (continuity).** Air at car speeds is practically incompressible, so whatever volume flows into a region must flow out again. Mathematically: the divergence of the velocity is zero, $\nabla \cdot \mathbf{U} = 0$.

Written out, these are the incompressible Navier–Stokes equations: three momentum equations (one for each velocity component u , v , w) and one continuity equation, for four unknowns u, v, w, p in every point.

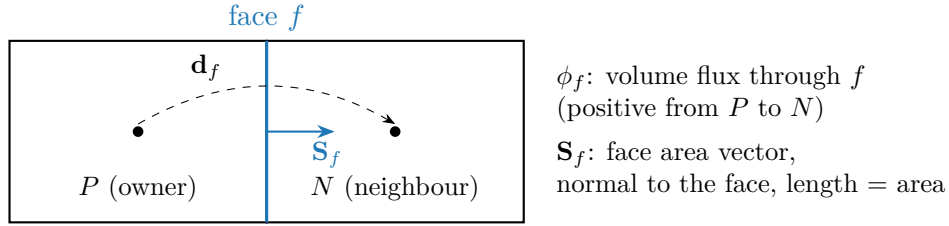


Figure 1: Two finite-volume cells sharing one face. Every balance in the solver is a sum of face contributions like this one.

RANS and the turbulence model. The real flow around a car is turbulent: it contains eddies over a huge range of sizes that fluctuate in time. A steady simulation does not resolve them. Instead it solves for the *time-averaged* flow (Reynolds-averaged Navier–Stokes, RANS) and represents the effect of the eddies by an extra “turbulent viscosity” ν_t that a turbulence model such as k - ω SST [19] computes from its own transport equations. For the purpose of this document, the turbulence model is a separate module that delivers ν_t ; coupledFoam leaves it exactly as it is in OpenFOAM (Section 4.18).

1.2 Why pressure is the awkward one

Look at the list of equations again. Each velocity component has “its” equation: the u -momentum equation. But which equation belongs to the pressure? The continuity equation $\nabla \cdot \mathbf{U} = 0$ does not contain p at all. Pressure has *no equation of its own*.

Pressure is instead whatever it has to be so that the velocity field that comes out of the momentum equations satisfies continuity. If too much fluid is heading into a region, the pressure there rises, pushes back, and diverts the flow until the mass balance holds. Mathematically, the pressure acts as a *constraint force* (a Lagrange multiplier) that enforces $\nabla \cdot \mathbf{U} = 0$.

Intuition: pressure as the enforcer

Think of a crowd walking through a building with fixed doors. Each person (fluid parcel) wants to keep going in their direction (momentum). The building cannot store people (continuity). Where the crowd would pile up, people start pushing back: that pushing is the pressure. Nobody decides the pressure in advance; it is simply whatever push is needed so that the number of people entering a room equals the number leaving it.

This is why incompressible solvers need a special strategy for pressure: there is no equation that you could simply “solve for p ”. The two strategies, segregated (Section 2) and coupled (Section 3), differ precisely in how they deal with this.

1.3 From equations to numbers: finite volumes

OpenFOAM is a *finite-volume* code. The domain is divided into cells (the mesh). For each cell the conservation laws are written as a balance over the cell: what enters through its faces minus what leaves equals the sources inside. Every face is shared by exactly two cells, the *owner* and the *neighbour* (Figure 1), or lies on the boundary. The quantity that crosses a face per second is the *face flux*; for mass it is the volume flux ϕ_f in m^3/s , the famous **phi** field of OpenFOAM.

Building block: finite volumes and faces

What is it? The domain is split into cells; each conservation law becomes “sum of what flows through the faces of a cell = sources in the cell”. Values live at cell centres, fluxes live

on faces.

Why do we need it? Face fluxes shared by two cells are counted once as outflow of one cell and once as inflow of the other, so mass is conserved exactly at the discrete level, whatever the mesh looks like. This is what makes the method robust on the unstructured `snappyHexMesh` meshes of real cars.

What would go wrong without it? Every quantity that `coupledFoam` builds, from the matrix to the Rhie–Chow flux, is a loop over faces. Nothing in this document works without this picture.

After discretisation the equations of all cells together form a big system of algebraic equations, one set of four equations per cell. Because each cell only talks to its face neighbours, most entries of the corresponding matrix are zero: the matrix is *sparse*. OpenFOAM stores such a matrix in its “LDU” format: a diagonal entry per cell and one “upper” and one “lower” entry per internal face.

1.4 Residuals: how far are we from the answer?

A steady solver cannot compute the answer in one go, because the equations are *nonlinear*: the velocity convects itself, so the coefficients of the equations depend on the solution. It iterates instead: guess, improve, improve again. After every iteration one plugs the current fields into the discrete equations and measures how badly they are violated. That imbalance, cell by cell, is the *residual*. At the exact discrete solution it is zero.

`coupledFoam` summarises the residual in one number R : the size of the imbalance of all four equations in all cells, normalised in the same way OpenFOAM normalises its residuals, and divided by its value in the first iteration. $R = 10^{-6}$ therefore means “the imbalance is a millionth of what it was at the start”. For comparison with `simpleFoam`, the log additionally prints r_U and r_p , computed exactly like the “Initial residual” that `simpleFoam` prints for U and p (D-016) [P: *Residual norms*].

Pitfall: residuals of different solvers are not directly comparable

R of `coupledFoam` and the initial residuals of `simpleFoam` are normalised differently. A residual plot with both solvers shows the *trend* of each, not which solver is “more converged”. That is why all timing comparisons in this project use a solver-independent criterion: the force coefficients (or the pressure drop) must stop changing (Section 6.1).

2 The standard approach: SIMPLE and simpleFoam

2.1 Guess and correct

The workhorse of OpenFOAM for steady incompressible flow is `simpleFoam`, based on the SIMPLE algorithm of Patankar and Spalding [24] or its variant SIMPLEC [30]. It is *segregated*: it never solves for velocity and pressure at the same time. One SIMPLE iteration does the following (Figure 2, left):

1. Take the current pressure as if it were correct, and solve the three momentum equations for a new velocity.
2. This velocity generally violates continuity. Derive, from the continuity equation and a simplified form of the momentum equations, a *pressure equation* (a Poisson-like equation) and solve it.

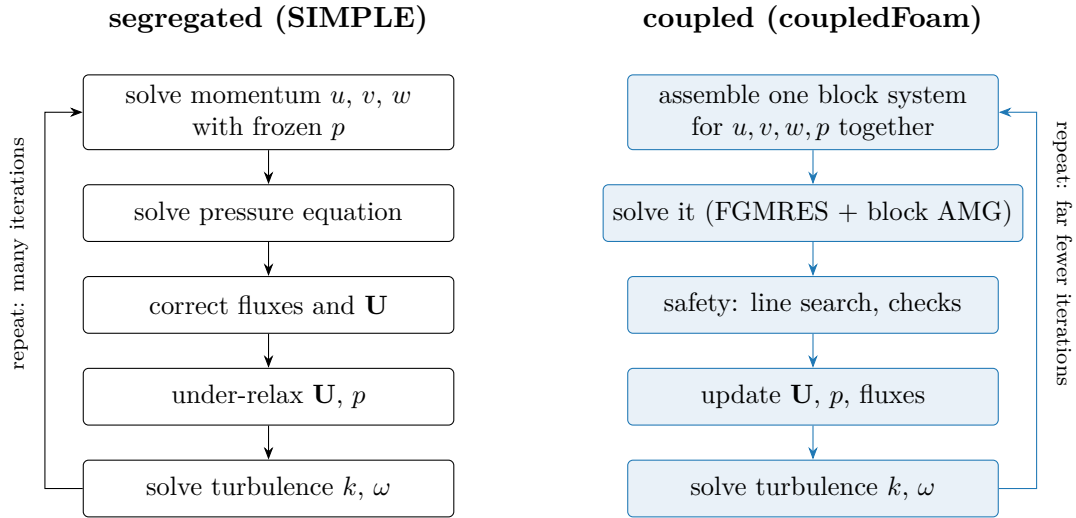


Figure 2: One outer iteration of the segregated SIMPLE algorithm (left) and of the coupled solver (right). In SIMPLE, velocity and pressure take turns; in coupledFoam they are solved together, and each iteration is more expensive but achieves much more.

3. Correct the face fluxes and the velocity with the new pressure gradient, so that the fluxes conserve mass.
4. Blend the new values with the old ones (under-relaxation), solve the turbulence equations, and start again.

2.2 Why SIMPLE needs under-relaxation

In step 2 the pressure equation is derived from a *simplified* momentum equation: the influence of the neighbouring velocities is neglected (SIMPLE) or approximated (SIMPLEC). The correction it produces is therefore too large or pointed in a slightly wrong direction. Applied at full strength it overshoots, the next iteration overshoots back, and the run oscillates or diverges. The cure is *under-relaxation*: accept only a fraction of the change (for example 0.3 for pressure and 0.7 for velocity in classical SIMPLE; SIMPLEC allows larger factors). The price is that every iteration makes only a fraction of the progress it could make.

Intuition: two people at one shower

Imagine two people adjusting the same shower: one controls the hot tap (velocity), the other the cold tap (pressure). They take turns, and each only feels the water temperature after the other has already turned his tap. If both turn boldly, the temperature swings back and forth. If both turn only a little each time (under-relaxation), the temperature settles, but it takes many rounds. A coupled solver is one person with both hands on both taps.

2.3 Why it takes so many iterations

In the segregated loop, information between pressure and velocity is exchanged only once per iteration, and only in relaxed doses. On a fine mesh, a pressure disturbance also has to travel across many cells, and the pressure equation is solved to a loose tolerance in each iteration. All this makes each iteration cheap but the number of iterations large, and it grows with the size of the mesh.

A concrete example from the test suite: on the lid-driven cavity at $Re = 100$ (test T0, Section 5.3), `simpleFoam` (SIMPLEC) needed 1932 iterations to reach its residual tolerance, while `coupledFoam` needed 58. (The two residuals are normalised differently, so this is an illustration of the order of magnitude, not a race; see Section 1.4.)

Why it matters: relaxation factors are case-specific

With `simpleFoam` you have probably spent time choosing relaxation factors: too small and the run crawls, too large and it diverges. The benchmark in this project therefore compares against two native settings: the tutorial’s own, and a tuned SIMPLEC setting with factors 1.0 for p and 0.9 for $\mathbf{U}$, k , ω (Section 6.1). `coupledFoam` has no relaxation factors; its step size is controlled automatically by the pseudo-time step and the line search (Sections 4.4 and 4.5).

3 The coupled idea

3.1 Solve everything together

A *coupled* solver discretises the three momentum equations and the continuity equation into *one* linear system and solves for u , v , w and p in all cells simultaneously. There is no pressure equation derived from a simplified momentum equation, and hence no need to under-relax the pressure: the pressure–velocity interaction is represented exactly (for the current linearisation) inside the matrix. This approach goes back to Vanka [31] and was developed for unstructured finite volumes by Darwish, Sraji and Moukalled [7, 8] and brought to OpenFOAM by Mangani, Buchmayr and Darwish [17, 18]. `coupledFoam` is a new, independent implementation of this idea for OpenFOAM v2606.

3.2 The 4×4 block structure

Because every cell has four unknowns (u, v, w, p), the matrix is naturally organised in small 4×4 *blocks*: one block on the diagonal for each cell (how the cell’s four equations depend on its own four unknowns) and one block for each face (how they depend on the neighbour’s four unknowns). Figure 3 shows what is inside a block.

Two things about this structure are worth remembering.

First, the bottom-right entry g (“pressure–pressure”) would be *zero* in a naive discretisation, because the continuity equation does not contain the pressure. A matrix with a zero block on the diagonal is called a *saddle-point* system; it is notoriously hard for iterative solvers. The Rhie–Chow interpolation (Section 4.3) is what fills this entry and stabilises the system.

Second, each block holds 16 numbers, where a segregated solver stores one number per cell and per face for each of its scalar equations. That is the memory price of coupling; Section 4.16 explains how `coupledFoam` reduces it by storing the blocks in single precision.

3.3 Outer and inner iterations

Even though velocity and pressure are solved together, `coupledFoam` still iterates, because the equations are nonlinear. There are two nested loops, and it helps to keep them apart:

- The **outer iteration** (also called nonlinear iteration): build the linear system from the current fields (convection uses the flux of the previous iteration), solve it, update the fields, update turbulence. This is the loop of Figure 2, and the “iterations” in the log are outer iterations.

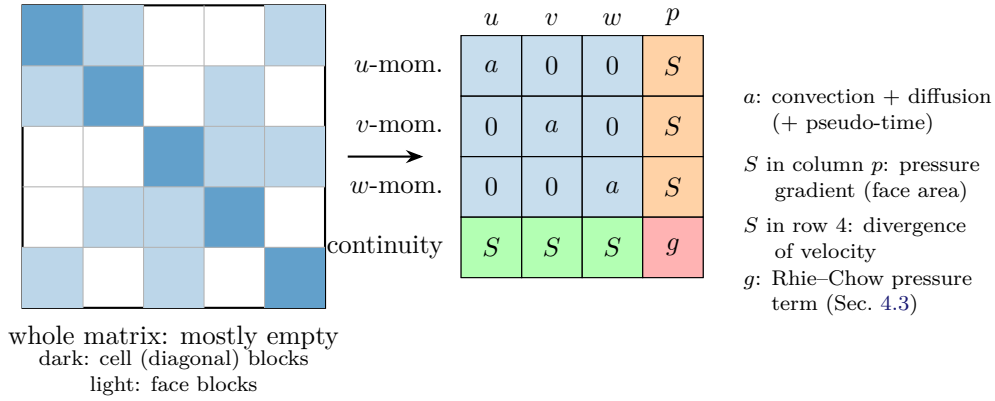


Figure 3: Left: the coupled matrix is sparse; each small square is a 4×4 block. Right: what a block contains. Rows 1–3 are the momentum equations, row 4 is continuity; columns are the unknowns u, v, w, p . The pressure column of the momentum rows (orange) and the velocity part of the continuity row (green) are mirror images: the pressure gradient and the velocity divergence are “transposes” of each other. The bottom-right entry (red) exists only because of the Rhie–Chow interpolation. (In rotating MRF zones the momentum part of the diagonal block also gets off-diagonal entries, Section 4.18.)

- The **inner iteration** (linear solve): inside each outer iteration, the big linear system itself is solved iteratively, by a Krylov method with a multigrid preconditioner (Sections 4.9–4.12). These are the “linear iterations” in the log.

3.4 The trade-off, and the only fair comparison

A coupled outer iteration does much more than a SIMPLE iteration, so far fewer of them are needed. But each one is more expensive: every coupling between two cells is a 4×4 block of 16 numbers instead of the single coefficients of SIMPLE’s scalar equations, the system is harder to solve than those separate scalar systems, and it needs more memory. Whether the coupled solver is *faster* in the end is therefore not obvious, and it can depend on the case.

Why it matters: wall-clock time and CPU-hours, not iterations

An iteration count says nothing about how long you wait or how much machine you use. A solver that needs 50 iterations of one minute each is slower than one that needs 2000 iterations of one second each. The project’s explicit requirement (D-025) is therefore: the fastest solver wins, measured as

- **wall-clock time** to a converged answer (how long you wait), and
- **CPU-hours** to a converged answer (how much computing you pay for: the CPU time of all ranks added up),

with the *same* convergence criterion for both solvers. Iteration counts are shown only for completeness.

3.5 A map of one outer iteration

Here is what coupledFoam does in every outer iteration, in plain words. Each step points to the building block that explains it.

1. Decide, for every cell, how large a pseudo-time step it may take (Section 4.4).

2. Assemble the momentum rows, including the convection scheme via deferred correction (Section 4.2) and the boundary conditions (Section 4.1); limit the time step in cells where the step would change too much.
3. Assemble the continuity row with the Rhie–Chow flux (Section 4.3); fix the pressure level in closed domains (Section 4.17).
4. Compute the residual in double precision, store the matrix in single precision (Section 4.16).
5. Solve the linear system for the *change* of the fields with FGMRES and block multigrid (Sections 4.9–4.12), to a tolerance chosen by Eisenstat–Walker (Section 4.13).
6. Check the proposed change: line search, cut the CFL number and repeat if the step is too wild or the solve failed (Section 4.5).
7. Apply the change (optionally accelerated by Anderson mixing, Section 4.14), update the face fluxes, check that nothing exploded, and roll back if it did (Section 4.7).
8. Update the problem-cell sets (Section 4.6), solve the turbulence model (Section 4.18), adapt the CFL number, write results and restart data (Section 4.8), test for convergence.

4 The building blocks

This section takes the solver apart. The order follows the map above: first how the equations are built, then how the nonlinear iteration is kept safe, then how the linear system is solved, and finally how single precision is made to work.

4.1 Boundary conditions and parallel interfaces

Every OpenFOAM boundary condition can express the boundary value as a linear function of the adjacent cell value: “boundary value = (some factor) $\times$ (cell value) + (some constant)”. For a fixed value the factor is zero; for zero gradient the factor is one and the constant zero. OpenFOAM computes these two coefficients for every patch type anyway. `coupledFoam` uses exactly these native coefficients to put every boundary condition *implicitly* into the block matrix (D-002). That is why `fixedValue`, `zeroGradient`, `slip`, `symmetry`, `inletOutlet`, `totalPressure` and conditions derived from them work without special code. For `inletOutlet` and `totalPressure`, the part that depends on the current flow direction or dynamic head is taken from the previous outer iteration. Wall functions act, as usual, through ν_t on the wall face.

When a case runs in parallel, the mesh is split into pieces (`decomposePar`); the cut faces become *processor* patches. In `coupledFoam` these are treated fully implicitly: the matrix entries across the cut are stored like any other face block and the neighbour values are exchanged during every matrix–vector product. Periodic (`cyclic`) and `cyclicAMI` patches are, in this version, coupled *explicitly*: the neighbour value from the previous outer iteration is used as a known source. That works, but converges more slowly across such patches; implicit coupling there is planned for Phase 2.

4.2 Convection schemes and deferred correction

Convection (the flow carrying momentum along) needs the velocity on the faces, but the unknowns live at cell centres. The simplest choice, *upwind*, takes the value of the upstream cell.

It is very robust, because it gives the matrix a strong diagonal, but it smears the solution (numerical diffusion). Higher-order schemes such as `linearUpwind` extrapolate from the upstream cell with its gradient; they are much more accurate but make the matrix less well behaved.

`coupledFoam` uses the standard trick of *deferred correction* [11, 15]: the matrix always contains the robust upwind operator, and the difference between your chosen scheme (from `fvSchemes`, key `div(phi,U)`) and upwind is added as a known source computed from the current fields. At convergence the source is exact, so the converged solution is that of your high-order scheme. A per-cell blending factor β between 0 (pure upwind) and 1 (full scheme) allows switching the correction off locally.

Building block: deferred correction and the upwind start-up

What is it? Matrix = upwind; right-hand side = $\beta \times (\text{high-order minus upwind})$, evaluated with the current fields.

Why do we need it? It keeps the linear system easy to solve while delivering the accuracy of the high-order scheme at convergence. For fresh runs, the first 50 iterations (or until $R < 10^{-2}$) use pure upwind ($\beta = 0$ everywhere), because the initial fields from `potentialFoam` are far from the answer and a high-order correction would only add noise. Cells with bad mesh quality or misbehaving values keep $\beta = 0$ (Section 4.6).

What would go wrong without it? Putting the high-order scheme directly into the matrix makes it much harder for the smoother and the Krylov solver; dropping the correction altogether gives an inaccurate, overly diffusive answer.

Pitfall: deferred correction at very large time steps

The correction is “one step behind”, and the pseudo-time term (Section 4.4) is what damps this lag. As the CFL number becomes very large, that damping disappears. A simple analysis in the professional paper [P: *Deferred correction at high CFL*] shows the iteration is still stable in idealised conditions, but with a small margin for high cell Péclet numbers. An inaccurate inner linear solve can use up that margin. This is part of the smoother story of Section 4.12.

4.3 Rhie–Chow interpolation and the checkerboard problem

On a mesh where velocity and pressure are stored at the same place (cell centres, as in OpenFOAM), a naive discretisation has a famous blind spot. Consider the pressure along a row of cells alternating between high and low: 100, 0, 100, 0, ... (Figure 4). The pressure gradient in a cell is computed from the face values, which are averages of the two neighbouring cells, so every face value is 50 and the gradient in every cell is zero. The momentum equations do not feel this wildly oscillating pressure at all, and nothing in the equations removes it. The resulting solutions show a “checkerboard” pattern in the pressure.

Rhie and Chow [25] fixed this in the definition of the face flux. The flux through a face is the interpolated velocity dotted with the face area, *plus* a correction proportional to

$$\underbrace{(p_N - p_P) \text{ measured directly across the face}}_{\text{sees the checkerboard}} - \underbrace{\text{average of the two cell pressure gradients}}_{\text{blind to it}}.$$

For a smooth pressure field the two terms are almost equal and the correction is tiny; for a checkerboard they differ strongly, and the correction creates a flux that drives the oscillation away. The size of the correction is set by a coefficient $D = V/\bar{a}$ (cell volume over the average momentum diagonal), which has the meaning “how easily does the velocity in this cell respond to a pressure difference”.

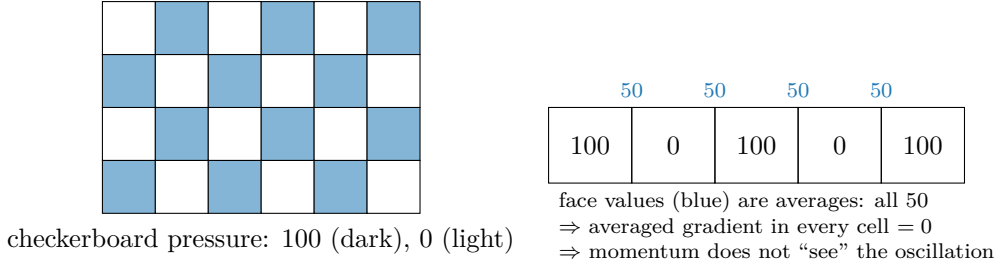


Figure 4: The checkerboard problem. With cell-centred storage and averaged face values, an alternating pressure field produces zero pressure force in every cell. The Rhie–Chow term compares the direct difference across each face ($100 - 0$) with the averaged gradient (0) and damps the mismatch.

In `coupledFoam` the direct-difference part is implicit and appears in the matrix as the pressure–pressure entry g of Figure 3; the averaged-gradient part is explicit. The continuity row *is* the sum of these Rhie–Chow fluxes over the faces of the cell, and after the solve the new face fluxes are computed with *exactly* the same face formula (D-003) [P: *Continuity row and Rhie–Chow interpolation*]. As a result, the mass imbalance of the fluxes that `coupledFoam` writes out is exactly the continuity part of its residual, and it vanishes as the solver converges.

Building block: Rhie–Chow interpolation

What is it? A correction of the face flux that compares the pressure difference across a face with the interpolated cell pressure gradients.

Why do we need it? It suppresses the checkerboard pressure mode and fills the empty pressure–pressure block of the coupled system, turning an unsolvable saddle point into a system that multigrid can handle.

What would go wrong without it? Oscillating, meaningless pressure fields, and a linear system with zeros on part of its diagonal that standard smoothers cannot even start on.

On boundaries the Rhie–Chow term is only used where the pressure is fixed (e.g. an outlet); on walls and velocity inlets it is switched off, since it would otherwise create an artificial flux through the wall (D-013).

Pitfall: the pseudo-time step is inside D

$D = V/\bar{a}$ includes the pseudo-time term of the next section. The converged solution therefore depends slightly on the final local time step through the Rhie–Chow term, in the same way a SIMPLE solution depends slightly on its relaxation factor. At the large CFL numbers reached at convergence the effect is small, and the cavity comparison with `simpleFoam` (Section 5.3) bounds it for that case.

4.4 Pseudo-transient continuation and the local CFL number

The steady problem is solved as if it were a time-dependent problem that is marched towards its steady state, but with a *fake* (“pseudo”) time step that is chosen for speed and robustness, not for accuracy. This is called pseudo-transient continuation (PTC) [14].

Technically, every momentum equation gets an extra term $V/\Delta t$ on its diagonal. A large diagonal makes the linear system easier to solve and makes each step smaller and more cautious. Crucially, the solver computes the *change* of the fields in each iteration (“increment form”, Section 4.16), with the current fields as the “old time” values. The pseudo-time term then only enters the matrix, never the residual: it controls *how* the solver walks, but not *where* it ends

up. The converged steady solution is the same whatever time steps were used (up to the small Rhie–Chow effect noted above).

Intuition: walking down a foggy mountain

You want to reach the bottom of a valley in fog. Small steps are safe but slow; big steps are fast but you might walk off a cliff. PTC is a rule for choosing the step length: start with small steps, and each time the ground has clearly gone downhill (the residual dropped), lengthen your stride. If a step looks dangerous, shorten it again. The valley floor (the steady solution) does not depend on how you walked there.

Local time steps and the CFL number. The time step is not the same everywhere. Each cell gets its own Δt from a *CFL number*: roughly, the number of cells a fluid parcel would cross in one time step. A small cell with fast flow gets a small Δt , a large cell in slow flow a large one, so that all cells progress at a similar “speed” towards the steady state. A diffusive limit is included for regions where viscosity dominates. The global CFL number starts at 5 and is adapted automatically between 1 and 500 (defaults).

How the CFL number is adapted. The default strategy (“mRDM” [5, 6]) looks at the residual: if it went down, the CFL number is multiplied by the ratio of the old to the new residual, at most by 1.5 per iteration; if it went up, the CFL number is kept. The CFL number is *reduced* only by the safety mechanisms (line search, failed linear solves, rollbacks), and after every reduction it is held for 10 iterations before it may grow again. Two alternative strategies (EXP: fixed growth factor; SER [21]: proportional to the residual ratio, may also decrease) are selectable.

Solution-limited local time step. Even with a moderate global CFL number, a few cells (say, at a sharp edge) may want to change much faster than the rest. Before the time step is applied, coupledFoam estimates the change each cell would undergo; if it exceeds half of the reference velocity, that cell’s time step alone is reduced. The global CFL can stay aggressive while individual trouble spots are throttled.

Building block: pseudo-transient continuation (PTC)

What is it? A fake time step per cell, $V/\Delta t$ on the diagonal, steered by an adaptive CFL number.

Why do we need it? It is the coupled solver’s replacement for under-relaxation: it makes early iterations cautious and late iterations bold, automatically. It also improves the linear system (larger diagonal) exactly when the flow is still far from converged.

What would go wrong without it? Solving the full steady coupled problem from a poor initial guess in one Newton-like jump usually diverges. With a fixed small time step it would be robust but as slow as a transient simulation.

Why it matters: reading the CFL column in the log

In a healthy run the CFL number climbs steadily towards its maximum and stays there. A CFL number that keeps dropping back, or never gets above a few tens, tells you the solver is repeatedly running into trouble: look at the line-search factor, the cut counter and the remediation set sizes in the same log line.

4.5 Line search and physicality checks

The linear solve proposes a change $\delta\mathbf{U}$, δp for every cell. Before it is applied, `coupledFoam` asks: *is this change physically reasonable?* The rule [6] is simple: no cell’s velocity may change by more than 30 % of a reference velocity U_{ref} (the largest velocity in the domain at the first iteration), and no cell’s pressure by more than half the reference dynamic pressure $p_{\text{ref}} = \frac{1}{2}U_{\text{ref}}^2$. If the proposed change is too large somewhere, the *whole* change is scaled down by a factor $\omega \leq 1$ (the line-search factor) until the rule holds.

If ω would have to be smaller than 0.1, the step is considered bad: the CFL number is halved and the step is recomputed from the same fields, up to three times. If it is still bad, the step is taken with $\omega = 0.1$, and the cells that caused the problem are marked for special treatment (Section 4.6).

A linear solve that failed (did not reduce its residual or produced non-finite numbers) is treated the same way: CFL cut and repeat (D-020, D-029). After three consecutive failed linear solves the run stops with a diagnostic output rather than carrying on with garbage.

Building block: line search

What is it? A factor $\omega \in (0, 1]$ that shortens the proposed update so that no cell changes by more than a fixed fraction of the reference velocity or pressure.

Why do we need it? The linear system is only a local approximation of the nonlinear problem; far from convergence its answer can be wildly wrong in a few cells. The line search keeps such a step from wrecking the fields.

What would go wrong without it? One bad step in a single cell (e.g. at a sharp corner) could create huge velocities, negative turbulence quantities, and a diverged run.

Pitfall: a failed solve must not be applied at all

During development a linear solve near the single-precision floor diverged, and the original rule “after the cuts, accept the step with $\omega = 0.1$ ” applied 10 % of a garbage increment to an already converged solution and destroyed it (D-020). Since then a failed solve is never applied, and the Krylov solvers return their best iterate rather than their last one.

Why it matters: what ω in the log tells you

$\omega = 1$ means the full step was taken. Values well below one in the early iterations are normal. Persistent small ω late in the run means some region keeps producing large changes; the dynamic remediation set size (next section) tells you how many cells are involved.

4.6 Remediation: special treatment for problem cells

Real meshes of real cars contain a few bad cells: highly non-orthogonal, skewed, stretched, or next to a much larger neighbour. And during a run, some cells may temporarily misbehave. `coupledFoam` handles both with *cell sets* that get gentler numerics locally, instead of making the whole simulation more cautious.

Static set (bad mesh). Built once from OpenFOAM’s mesh-quality measures: non-orthogonality above 70° , skewness above 4, a volume ratio to a neighbour above 20, or aspect ratio above 1000. These cells get pure upwind convection, a stronger limiter on the non-orthogonal correction, a limited pressure gradient in the Rhie–Chow term, and half the time step.

Dynamic set (bad behaviour). Updated every iteration. A cell is added if its velocity ex-

ceeds twice the reference, its pressure exceeds five times the reference, it differs strongly from its neighbours, any value is not a finite number, or the line search or the rollback (Section 4.7) points at it. The cell and its direct neighbours then get pure upwind, a tenth of the time step, and a clip on how far the update may move them away from their neighbours' mean. A cell leaves the set after 20 quiet iterations. A warning appears if the sets cover more than 1 % of the mesh.

Zonal factors (optional). You can name cell zones or patches (with a number of cell layers around them) where the time step and the high-order blending should be reduced, e.g. around a wheel. Off by default (D-027).

Intuition: local speed limits

Rather than lowering the speed limit on the whole motorway because of one construction site, you put a local speed limit at the construction site. Remediation is exactly that: local time step and scheme reductions where they are needed, full speed everywhere else.

Why it matters: what the remediation sets mean for accuracy

Pure upwind in a handful of cells changes the solution only locally and slightly. The test criteria require the static set to stay at or below 1 % of the cells on the motorBike meshes (T4), and an empty dynamic set at convergence on the airfoil (T3). If on your case a large fraction of cells ends up in the sets, that is a sign to improve the mesh.

4.7 Sentinel and rollback

Before every field update, `coupledFoam` keeps a copy of the fields. After the update (and again after the turbulence update), a “sentinel” checks three things: are all values finite numbers, is the largest velocity below 10 times the reference, and is the largest pressure below 50 times the reference? If not, the copy is restored (a *rollback*), the CFL number is cut to a quarter, and the offending cells go into the dynamic set. After five consecutive rollbacks the run stops, writes the last valid fields into a time directory `<iteration>_lastValid` together with a field that marks the offending cells, prints their positions, and exits with an error. A time directory containing `nan` or `inf` is never written.

Why it matters: when a run stops

If `coupledFoam` stops with a sentinel error, open the `_lastValid` time directory in ParaView and look at the `sentinelFlag` field: it shows you where the trouble started, usually at a mesh defect or a boundary-condition problem.

4.8 Restart

Whenever results are written, `coupledFoam` writes the normal OpenFOAM fields and, last, a small dictionary `coupledState` with everything the iteration needs to continue exactly where it stopped: iteration number, CFL number, residual history values, reference values, the dynamic set, the rollback counters and more (D-017). It is written under a temporary name and renamed at the end, so its presence guarantees that the time directory is complete. On restart, the face fluxes are recomputed from the stored velocity and pressure with the stored Rhie–Chow coefficient, and the difference to the stored fluxes is logged as a consistency check. The restart test (T-restart) requires that a run split in two reaches the same answer as a continuous run. The optional Anderson history (Section 4.14) is not part of the restart state (D-026): a restarted run begins with an empty history.

4.9 Krylov solvers: GMRES, FGMRES and BiCGStab

Now to the inner loop: solving the big linear system $\mathbf{Ax} = \mathbf{b}$ once per outer iteration. For millions of cells a direct solution (Gaussian elimination) is out of the question in memory and time. Iterative *Krylov* methods [27] are used instead.

Intuition: building the answer from repeated experiments

A Krylov method only needs one operation: “multiply a vector by the matrix”. It starts with the current error (the residual), multiplies it by the matrix, multiplies the result again, and so on. Each product shows how the system “responds” to a certain pattern. From these responses it builds a combination that best explains the right-hand side. Think of tuning a sound system: you play a few test signals, record how the room responds, and combine them into the correction that best cancels the echo.

GMRES (Generalised Minimal RESidual) keeps all the directions it has generated and picks, at every step, the combination with the smallest residual. It never gets worse, but the memory grows with every step, so it is *restarted* after m steps (default $m = 10$; $m = 6$ above 40 million cells to save memory).

FGMRES (Flexible GMRES [26]) is GMRES that additionally stores the preconditioned directions. This costs twice the memory but allows the preconditioner to be slightly different every time it is applied, which is the case for the K-cycle multigrid (Section 4.11). FGMRES is the default in coupledFoam, and the K-cycle is refused at start-up with any other Krylov solver.

BiCGStab (BiConjugate Gradient Stabilised) needs only a few vectors regardless of the number of steps, but its residual does not decrease monotonically and it can break down. coupledFoam’s version restarts after a breakdown and returns the best iterate it has seen.

All inner products and norms inside these solvers are accumulated in double precision, even though the vectors are stored in single precision (Section 4.16).

Building block: Krylov solver

What is it? An iterative method that improves the solution of $\mathbf{Ax} = \mathbf{b}$ using only matrix–vector products and combinations of vectors.

Why do we need it? It is the only affordable way to solve linear systems with tens of millions of unknowns, and it parallelises well.

What would go wrong without it? Direct solvers would need far more memory than the machine has for any realistic mesh.

4.10 Preconditioning

On its own, a Krylov method converges slowly on the coupled system: the matrix mixes scales from single cells to the whole domain, and a handful of matrix–vector products cannot capture that. A *preconditioner* $\mathbf{M}$ is a cheap, approximate inverse of $\mathbf{A}$. Instead of the original system, the Krylov method solves one in which $\mathbf{A}$ is combined with $\mathbf{M}^{-1}$; if $\mathbf{M}$ is a good approximation, this combined system is close to the identity and is solved in a few steps.

Intuition: glasses for the solver

The Krylov method is a good reader with bad eyes. The preconditioner is a pair of glasses: it does not read for you, but it makes the text sharp enough that reading is fast. Better glasses (a stronger preconditioner) cost more to make, so there is a sweet spot.

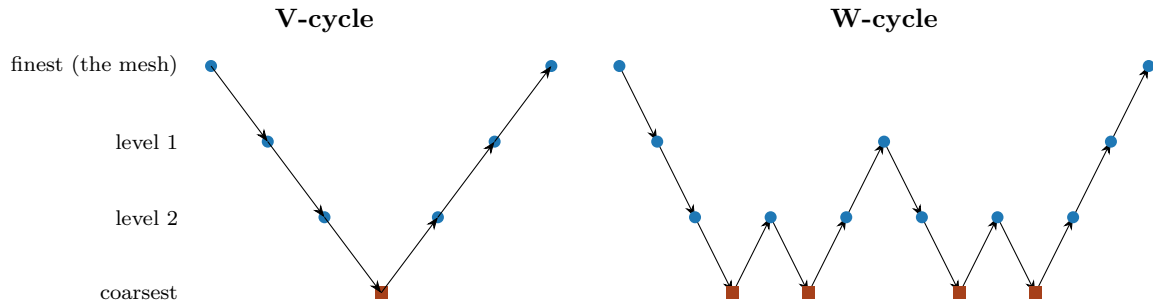


Figure 5: Multigrid cycles over four levels. Going down: smooth, then pass the remaining residual to the coarser level. Going up: add the coarse correction, smooth again. Blue dots: smoothing; orange squares: coarsest-level solve. The V-cycle visits each coarse level once; the W-cycle visits coarse levels repeatedly, which is stronger but more expensive. The F-cycle lies in between; the K-cycle decides at every level and every visit whether a second, W-like visit is worth it.

coupledFoam offers two preconditioners: `blockDiagonal` (invert only the 4×4 diagonal block of each cell; cheap and weak; used in the benchmark to measure how much the multigrid is worth) and `blockGAMG`, the block algebraic multigrid described next (the default).

4.11 Algebraic multigrid

The key observation. Simple iterative “smoothers” such as Gauss–Seidel (Section 4.12) update each cell from its neighbours. They remove *jagged* errors (errors that change sign from cell to cell) very quickly, because those are local. But *smooth* errors, spread over hundreds of cells, are reduced only very slowly: information travels one cell per sweep.

The key trick. A smooth error on a fine mesh looks jagged on a much coarser mesh. So: smooth a little on the fine mesh, then transfer the remaining (smooth) error problem to a coarser mesh, where it is cheap to reduce, and recursively to even coarser meshes; then interpolate the correction back and smooth again. That is *multigrid*.

Intuition: fixing a picture at several zoom levels

To fix the colour balance of a large photo you do not retouch pixel by pixel: you fix the overall tone on a thumbnail, then the regional shading at medium size, then the fine details at full size. Each level corrects the errors it can see best.

Algebraic and additive-correction. OpenFOAM’s `GAMG` and coupledFoam’s `blockGAMG` build the coarse “meshes” purely from the matrix and the geometry of the faces, without a second mesh file: neighbouring cells are glued together into coarse cells (*agglomeration*), using OpenFOAM’s own `faceAreaPair` agglomeration. In the additive-correction approach [13, 33], the equation of a coarse cell is simply the sum of the equations of its fine cells, and the coarse correction is added unchanged to all fine cells it contains. coupledFoam does this with 4×4 blocks; the sums are accumulated in double precision and the result is stored in single precision. With `mergeLevels` 2, two pairwise agglomeration steps are merged into one level, so each level has roughly a quarter of the cells of the one above.

Cycle types. How often the coarse levels are visited is the *cycle type* (Figure 5):

V once per level: cheapest.

W twice per level, recursively: strongest and most expensive.

F in between: one extra visit per level.

K (Notay and Vassilevski [23], the basis of the AGMG solver [22]; default): at each coarse level, the correction is treated as a mini-Krylov step; if the first visit reduced the coarse residual well (to a quarter or better), it stops there (V-like); otherwise it makes a second visit and optimally combines the two (W-like). It thus adapts per level and per application.

Additive-correction multigrid has a comparatively weak coarse-level correction, which is why a plain V-cycle does not scale optimally with it, and why the adaptive K-cycle is the default. Because the K-cycle is not exactly the same operation every time it is applied, it needs the flexible Krylov solver FGMRES.

What cycles cost. If each level has r times fewer cells than the one above, the work of one cycle, measured in units of “one sweep on the finest mesh”, is (derived in the professional paper [P: *Block additive-correction AMG*]):

coarsening ratio r	2	3	4	8
V	2.00	1.50	1.33	1.14
F	4.00	2.25	1.78	1.31
W	∞	3.00	2.00	1.33

With plain pairwise agglomeration ($r = 2$) a W-cycle would cost more with every additional level. The solver therefore enforces a rule: for W- and K-cycles every level must have at least three times fewer cells than the one above; if not, it merges more agglomeration steps (up to four) and tries again, and stops with an error if the rule still fails. It also checks that the total size of all coarse matrices stays below 1.5 times the fine matrix (“operator complexity”).

The coarsest level. At the bottom of the hierarchy the system is small. If it has at most 64 cells on one rank, it is solved exactly by a dense LU factorisation in double precision; otherwise by a few BiCGStab iterations. The choice is automatic and printed in the log.

Building block: algebraic multigrid (block-GAMG)

What is it? A hierarchy of ever coarser versions of the matrix, built by gluing cells together, with smoothing on each level and corrections passed between levels.

Why do we need it? It reduces errors of all sizes at a cost proportional to the number of cells. It is what makes the solver usable on meshes with millions of cells.

What would go wrong without it? With a simple preconditioner (block diagonal), the number of Krylov iterations grows strongly with mesh size; configuration D of the benchmark measures exactly this (Section 6.1).

Cycle-efficiency controller (autoTune). The solver measures how well the multigrid cycle works: on the first application in each linear solve it checks by how much one cycle reduces the residual (the ratio ρ ; small is good). Every 50 outer iterations it looks at the median ρ . If the cycle is weak ($\rho > 0.7$ in two consecutive windows) it adds post-smoothing sweeps (up to 4) and, for V and F cycles, promotes the cycle type (V→F→W, only if the coarsening rule allows); if the cycle is very effective ($\rho < 0.3$) it removes sweeps again. Changes happen only between linear solves and are logged as events. The K-cycle is not promoted further, since it adapts internally.

4.12 Smoothers: why Gauss–Seidel failed and ILU(0) worked

The smoother is the local workhorse of each multigrid level. `coupledFoam` has two:

Block Gauss–Seidel. Go through the cells one by one; for each cell, solve its own 4×4 block exactly, using the latest values of the neighbours. Cheap and simple.

Block ILU(0) (incomplete LU factorisation without fill-in, in OpenFOAM’s “DILU” form, D-015). Build an approximate factorisation of the whole matrix that keeps the original sparsity pattern and only modifies the diagonal blocks, then apply it in one forward and one backward sweep. More expensive to set up, but it transports information along the whole sweep direction and treats the coupling between cells much more faithfully.

A learning story (D-028). During development, the cavity at $Re = 1000$ with the `linearUpwind` scheme refused to converge with the original configuration (BiCGStab, block multigrid, block Gauss–Seidel smoother): as soon as the upwind start-up ended and deferred correction was switched on, the CFL number oscillated, the line search kept cutting it, and the residual stagnated far above the target. With pure upwind the same configuration converged. The obvious suspect was deferred correction at large time steps, and an analysis (Section 4.2, pitfall box) showed that the theoretical stability margin is indeed small.

The decisive experiment was different: *keep everything the same and only change the smoother to block ILU(0)*. The same discretisation then reached the maximum CFL number without a single cut and converged to $R < 10^{-8}$ quickly, with a fraction of the Krylov iterations and of the solve time. The real culprit was the *quality of the inner solve*: the 4×4 point smoother is weak on the saddle-point structure of the coupled system, the resulting poor increments were what made the outer iteration oscillate, and deferred correction merely had too little margin to absorb them. The measured details are in D-028 and in the professional paper [P: *Deferred correction at high CFL*].

Why it matters: what this story teaches

- Symptoms in the outer iteration (oscillating CFL, stagnating residual) can be caused by the inner linear solver. Before changing the discretisation, check whether the linear solves reach their tolerance.
- The accuracy of the inner solve and the robustness of the outer iteration have to be designed together. This is why `coupledFoam` has the Eisenstat–Walker tolerance (Section 4.13), stronger cycles, and the treatment of failed solves.

Current status of the default. The formal decision on the default smoother is made by a dedicated benchmark on the backward-facing step (T2, gate B2 of the specification: ILU(0) must bring at least 15% wall-clock gain). Until that is run, the test cases still name `blockGaussSeidel`; the recommended settings for the F1 case (Section 7) already use `blockILU0`.

4.13 Eisenstat–Walker: do not over-solve

How accurately should the linear system be solved in each outer iteration? Early on, the matrix itself is only a rough approximation of the nonlinear problem (it was built from fields that are still far from the answer). Solving it to ten digits is wasted effort: the next outer iteration builds a different matrix anyway. Late in the run, when the outer iteration converges fast, a sloppy linear solve would slow it down.

The Eisenstat–Walker rule [10, 16] sets the linear tolerance η_n (the factor by which the linear residual must be reduced) from the progress of the outer iteration: roughly $\eta_n \approx 0.9 (R_n/R_{n-1})^2$. When the outer residual drops quickly, η_n becomes small (solve more accurately); when it stagnates, η_n stays large (solve loosely). It is kept between 10^{-3} and 0.5, is 0.5 in the first iteration and during the upwind start-up, and at least one Krylov iteration is always made.

Intuition: measuring with the right ruler

When you are still sawing a board roughly to length you do not measure with a micrometer; you measure to the millimetre only for the final cut. Eisenstat–Walker picks the measuring precision of each inner solve to match how close the outer iteration is to the answer.

Pitfall: a zero increment freezes the solver

If the linear solver’s absolute tolerance is reached before the outer target, the solver may decide to do zero iterations, the increment is zero, and the outer iteration stops moving. This happened on the cavity (D-022). Hence the rule “at least one Krylov iteration”, and the requirement that the absolute linear tolerance lie below the outer target.

4.14 Anderson acceleration (optional)

Anderson acceleration [2, 32] looks at the last few outer updates (default four) and asks: is there a combination of them that would have predicted the current one better? If the updates follow a pattern (for example, they keep zig-zagging), it extrapolates along that pattern and takes a larger, better-aimed step. The extrapolated state must pass the same sentinel checks; the history is cleared after every CFL cut, every change of the remediation sets and every rollback.

It is *off by default*, because it needs to store the recent history: about 320 bytes per cell for four steps, which matters at tens of millions of cells. The benchmark measures whether it pays off on T1 and T3 (Figure 20).

4.15 Processor agglomeration: gathering coarse levels

In a parallel run each rank owns a piece of the mesh. On the fine levels each rank has plenty of work, but a few levels down each rank owns only a handful of coarse cells, and the time is dominated by waiting for messages from other ranks (*latency*), not by arithmetic. coupledFoam therefore gathers coarse levels onto fewer ranks: a level with fewer than 5000 cells per rank is moved onto a quarter of the ranks, and a level with fewer than 5000 cells in total onto one rank (D-030). The finest level is never gathered.

Intuition: do not hold a ten-person meeting about three items

If ten people each have one small item to decide, calling a meeting where everyone waits for everyone else costs more than letting one person collect the items and decide alone. Skipping the coarse levels altogether is not an option: they carry the large-scale correction and cost less than 5 % of the cycle work.

4.16 Float versus double: the precision design

Computers store real numbers in *floating point*: double precision (“double”, 8 bytes, about 16 significant digits) or single precision (“float”, 4 bytes, about 7 digits). OpenFOAM normally works entirely in double.

Why float at all? Sparse linear algebra is limited by *memory bandwidth*: the processor spends most of its time waiting for matrix entries to arrive from memory, not computing with them. A float matrix is half the size, so it can be streamed through the processor in roughly half the time. The coupled matrix needs 16 numbers per cell for the diagonal block and 32 per internal face for the two off-diagonal blocks; on a hexahedral mesh with about three internal faces per cell that is about 448 bytes per cell in float, half the double-precision figure. Memory size matters just as much: the budget for the largest target mesh (45 million cells) is set out in the professional paper [P: *Storage and precision*].

Why not float everywhere? Seven digits are not enough to add up millions of numbers accurately, and not enough to reach the residual levels a converged steady solution needs. So coupledFoam separates two roles (D-001, D-012):

- **Stored in float (blockScalar)**: the block matrix and all Krylov and multigrid vectors, i.e. the big, bandwidth-hungry data.
- **Computed in double (reduceScalar)**: every sum, dot product and norm (accumulated in double, reduced across ranks in double), the Galerkin sums of the coarse levels, the small dense matrices of GMRES, and above all *the residual* $\mathbf{b} - \mathbf{Ax}$, which is computed from the double-precision assembly before the matrix is rounded to float.

Why this works: the increment form. In each outer iteration the solver does not solve for the new fields, but for the *change* $\delta\mathbf{x}$: $\mathbf{A}_{\text{float}} \delta\mathbf{x} \approx \mathbf{r}$, with the residual $\mathbf{r}$ computed in double. The float solve only has to get the change roughly right; the next iteration measures the remaining error exactly again, in double, and corrects it. The final accuracy is therefore set by the double-precision residual, not by the float solve. This is the classical idea of mixed-precision iterative refinement [3, 4], built into the outer iteration.

Intuition: a cheap tape measure and a precise reference

You adjust a machine part using a cheap tape measure (the float solve), but after each adjustment you check the part against a precise gauge (the double residual). The cheap tape is good enough to know which way and roughly how far to adjust; the precise gauge guarantees you do not stop until the part is exactly right.

Pitfall: why OpenFOAM's own mixed-precision build did not help

The specification asked for OpenFOAM's SPDP build. It turned out that in OpenFOAM v2606 this build does the opposite of what is needed: it stores the *fields* in float and *solves* in double (D-001). coupledFoam therefore runs on a normal double-precision OpenFOAM and defines its own float and double types for the block data. The native `simpleFoam` reference thus uses the same double-precision fields.

Pitfall: adding many small numbers in float

Summing 10^8 copies of 10^{-4} in float gives a visibly wrong result, because each small addition to a large running sum is rounded away. The unit test `Test-doubleReduce` demonstrates this and checks that the double accumulator is exact (D-004). Every reduction in the solver therefore goes through the double-precision layer.

Other safety measures of the precision design: the 4×4 block inversions are done in double with a guard against zero pivots, and any value too large for float ($|v| > 10^{30}$) is clamped and counted during the narrowing; a non-zero count is a warning in the log and a failure in the tests.

Building block: mixed precision

What is it? Store the big matrix and solver vectors in float; compute residuals, sums and reductions in double.

Why do we need it? Halves the memory traffic and matrix memory of the most expensive part of the solver, without losing final accuracy.

What would go wrong without it? All-double: twice the matrix memory and more time per linear iteration. All-float: the solver would stall at a residual level far above what a converged steady solution needs, and sums over large meshes would be wrong.

4.17 Fixing the pressure level in closed domains

In a closed domain (no boundary fixes the pressure, e.g. the cavity), only pressure *differences* matter: adding a constant to p everywhere changes nothing. The linear system then has a “free direction” that has to be pinned down. OpenFOAM’s usual weak reference only nudges one cell. In double precision that is enough, but once the right-hand side is rounded to float the float Krylov solve starts drifting along the free direction, and the run failed on the cavity (D-021). `coupledFoam` therefore *replaces* the continuity equation of the reference cell by “ p in this cell equals the reference value”. This is exact, because in a closed incompressible domain the continuity equations of all cells add up to zero, so one of them is redundant. For open domains with a fixed-pressure outlet, such as an external car simulation, this does not apply.

4.18 Turbulence, MRF and the rest

Turbulence. The turbulence model is *not* part of the coupled block. After each coupled update, `coupledFoam` calls the unmodified OpenFOAM turbulence model once (`turbulence->correct()`), with its usual solvers and relaxation factors for k and ω . Afterwards k and ω are kept above small minimum values and ν_t is capped at 10^5 times the molecular viscosity (the number of capped cells is logged). Any OpenFOAM incompressible RANS model works; the tests use k - ω SST [19] and GEKO [20].

MRF (rotating zones). Multiple-reference-frame zones, as used for rotating wheels, add a rotation term to the momentum equations. In `simpleFoam` this term is explicit; `coupledFoam` puts it into the 3×3 momentum part of the diagonal block, i.e. implicitly (D-006), extracted from OpenFOAM’s own MRF operator so that any number of zones is handled. As in `simpleFoam`, the flux is stored relative to the rotating frame inside MRF zones (D-014).

Non-orthogonal meshes. On skewed, non-orthogonal meshes the diffusion terms and the Rhie–Chow pressure term need a correction. It is applied explicitly with a limiter (default 0.5, and 0.2 on faces of static-remediation cells), in the same spirit as OpenFOAM’s `limited corrected` scheme.

5 The test cases

5.1 Primer: how to read the results

How to read a convergence plot. Convergence plots (Figure 6) show a residual against the iteration number with a *logarithmic* vertical axis: each grid line is a factor of ten. On such an axis, steady exponential convergence is a straight line going down; the steeper, the faster.

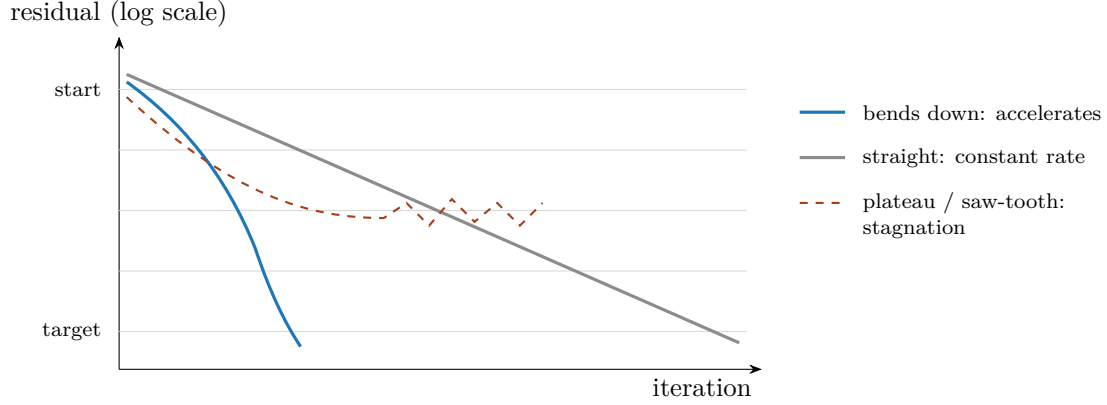


Figure 6: Illustrative sketch (not data) of three typical residual histories on a logarithmic axis.

- A *straight falling line* is healthy convergence.
- A curve that *steepens* towards the end (bends downwards) is the typical signature of a solver that gets more efficient as it approaches the solution, as Newton-like methods do.
- A *flat line* (plateau) means stagnation; a *saw-tooth* or noise band means the iteration is oscillating.
- Spikes that recover quickly are often a CFL cut or a rollback; compare with the CFL panel.

Remember that the residual levels of different solvers are normalised differently (Section 1.4); compare shapes, not heights.

What are C_d and C_l ? The force the air exerts on a body is split into *drag* F_D (along the free-stream direction) and *lift* F_L (perpendicular to it; for a race car the interesting part is negative lift, i.e. downforce). They are made dimensionless with the dynamic pressure of the free stream and a reference area A :

$$C_d = \frac{F_D}{\frac{1}{2}\rho U_\infty^2 A}, \quad C_l = \frac{F_L}{\frac{1}{2}\rho U_\infty^2 A}.$$

In OpenFOAM they are computed by the `forceCoeffs` function object. In a steady RANS run they settle to constant values as the solver converges; the benchmark uses exactly this settling as its convergence criterion (Section 6.1).

Relative L_2 difference. To compare two profiles (say, the velocity along a line from coupledFoam and from simpleFoam), one takes the root of the sum of squared differences at all points, divided by the root of the sum of squared reference values. A value of 10^{-3} means the curves differ by about a tenth of a per cent of their typical size; a value of 10^{-6} means they are identical for all practical purposes.

What does “pass” mean? Every test has pass criteria fixed in the specification before any result was known, and they may only be tightened, never relaxed. The main criterion is always a comparison with `simpleFoam` on the *identical* mesh, schemes and boundary conditions: both solvers solve the same discrete equations, so their converged answers should agree closely. Agreement with experiments is reported where available but is a property of the turbulence model and the mesh, not of the solver.

Table 1: Test results (spec 13).

test	pass	iterations	final R	wall [s]	CPU-h	key metric
T0_Re1000_np1	yes	58	7.35e-09	17.9	n/a	l2rel_u = 1.43e-05
T0_Re100_np1	yes	58	5.25e-09	9.62	n/a	l2rel_u = 2.89e-06
Test-block4Ops	yes	n/a	n/a	n/a	n/a	maxInverseError = 6.24e-08
Test-blockGAMG	yes	n/a	n/a	n/a	n/a	crossRankRelDiff = 1.08e-06
Test-blockGAMG_np1	yes	n/a	n/a	0.132	n/a	
Test-blockGAMG_np4	yes	n/a	n/a	0.136	n/a	
Test-blockMatrix	yes	n/a	n/a	n/a	n/a	crossRankRelDiff = 1.71e-09
Test-blockMatrix_np1	yes	n/a	n/a	n/a	n/a	
Test-blockMatrix_np4	yes	n/a	n/a	n/a	n/a	
Test-doubleReduce	yes	n/a	n/a	n/a	n/a	
Test-doubleReduce_np1	yes	n/a	n/a	n/a	n/a	relErrorDouble = 0.00e+00
Test-doubleReduce_np4	yes	n/a	n/a	n/a	n/a	relErrorDouble = 0.00e+00
Test-precision	yes	n/a	n/a	n/a	n/a	
test_env	yes	n/a	n/a	n/a	n/a	

5.2 Test protocol

Unless stated otherwise, every case is run on 1 and on 4 parallel ranks, and both must pass, with integral quantities agreeing between rank counts to 10^{-4} relative. Development runs trap floating-point exceptions (a division by zero or an invalid operation aborts the run); any trap is a failure. Below the case tests sit unit tests of the individual components (block inversion, block matrix product against OpenFOAM’s own, double reductions, the multigrid on a model problem, FGMRES with a deliberately changing preconditioner). Table 1 lists all test records produced so far.

How to read Table 1. One row per test record (`_np1`, `_np4`: run on 1 or 4 ranks). “pass” is the verdict of the test’s own criteria; “iterations” and “final R” refer to the outer iteration of `coupledFoam`; “wall” is the elapsed time of the run on a *shared* development machine and is not a benchmark result; “key metric” shows the most important measured quantity of the test, e.g. `l2rel_u` (relative L_2 difference of the u profile against `simpleFoam`) or `crossRankRelDiff` (difference between the 1- and 4-rank solutions).

5.3 T0: lid-driven cavity

What it is. A square box filled with fluid whose top wall (the lid) slides sideways; the fluid is dragged along and forms a large rotating vortex, with small counter-rotating vortices in the corners. Laminar, 2D, 128×128 cells, at Reynolds numbers 100 and 1000, with the `linearUpwind` scheme. It is the classical first test of any incompressible solver; reference data by Ghia et al. [12].

What it tests. The core of the solver: the block assembly, Rhie–Chow, the closed-domain pressure reference, the precision design, and whether a float linear solver can reach a residual of 10^{-8} . Pass: $R < 10^{-8}$ within 300 iterations, centreline profiles within 10^{-3} relative L_2 of `simpleFoam`, no floating-point trap, no clamped coefficients.

Result at $Re = 100$. `coupledFoam` converged to $R < 10^{-8}$ in 58 outer iterations; `simpleFoam` needed 1932 iterations to its own tolerance of 10^{-8} . The centreline profiles agree to a relative L_2 difference of 2.9×10^{-6} in u and 5.5×10^{-6} in v , far below the threshold of 10^{-3} . Two solvers that solve the same discrete equations have arrived at the same answer.

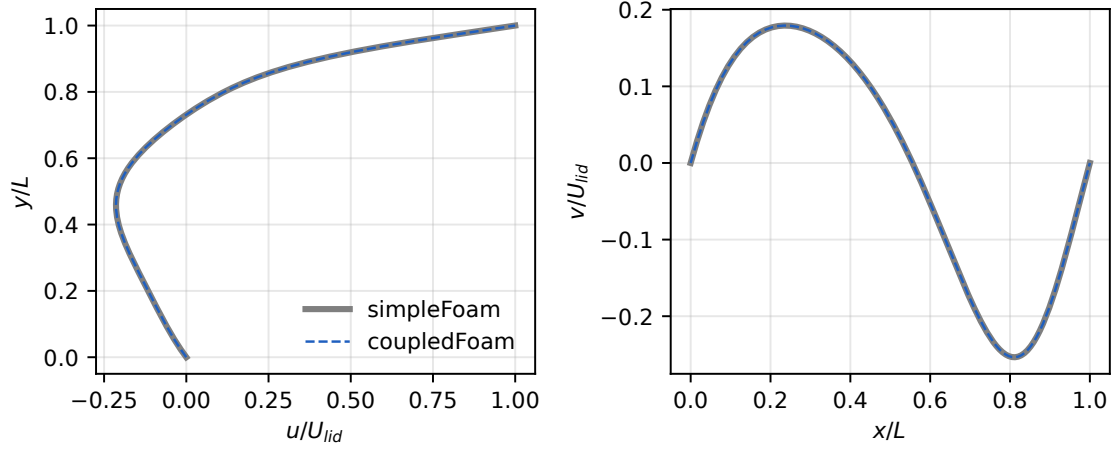


Figure 7: T0, lid-driven cavity at $Re = 100$: centreline profiles $u(y)$ and $v(x)$, coupledFoam against simpleFoam.

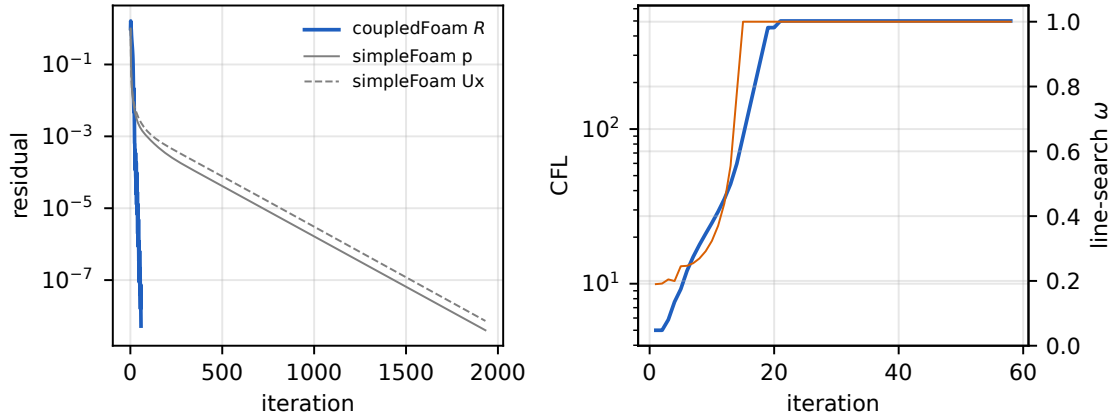


Figure 8: T0, $Re = 100$, one rank: combined residual R of coupledFoam and native initial residuals of simpleFoam (left); CFL number and line-search factor ω (right).

How to read Figure 7. Left: the horizontal velocity u (divided by the lid speed) along the vertical line through the centre of the box, plotted against height. Near the top it approaches the lid speed; lower down it is negative, which is the return flow of the vortex. Right: the vertical velocity v along the horizontal centreline. `simpleFoam` is drawn as a thick line, coupledFoam as a thin dashed line on top of it. *Good* means: you cannot see two lines.

How to read Figure 8. Left: residuals against iteration on a log axis; coupledFoam's R together with the p and U_x initial residuals of `simpleFoam`. Look at the *shapes*: the coupled residual falls steeply within few iterations, the native ones decrease slowly over far more iterations. Right: the CFL number (log axis) ramps up from its start value to the maximum, and the line-search factor ω (right axis, between 0 and 1) shows whether full steps were taken. A smooth CFL ramp and ω at one are the picture of an untroubled run.

$Re = 1000$ is the harder variant: the vortex is stronger, the corner vortices are larger and the convection scheme matters more. This is the case of the smoother story (Section 4.12). Figures 9 and 10 are read in the same way as those for $Re = 100$; the measured values are listed

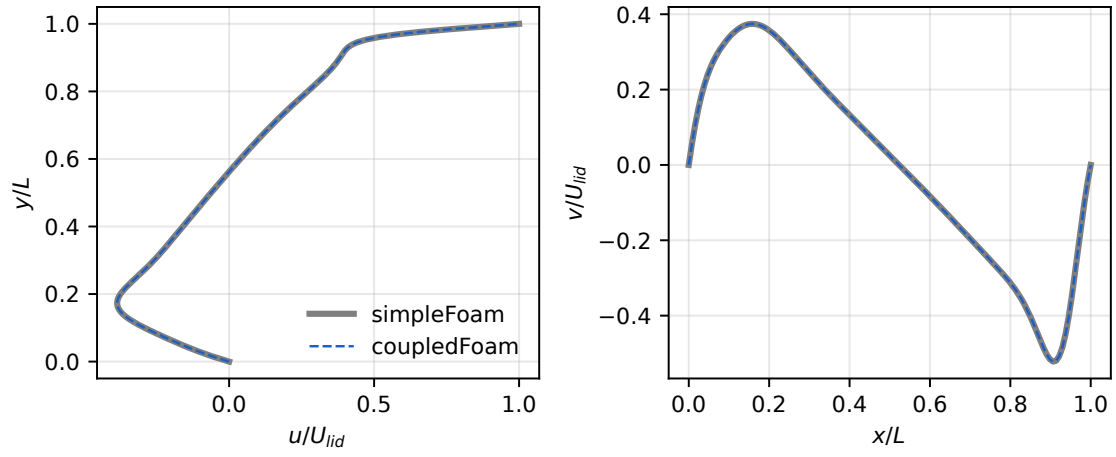


Figure 9: T0, lid-driven cavity at $Re = 1000$: centreline profiles, coupledFoam against simpleFoam.

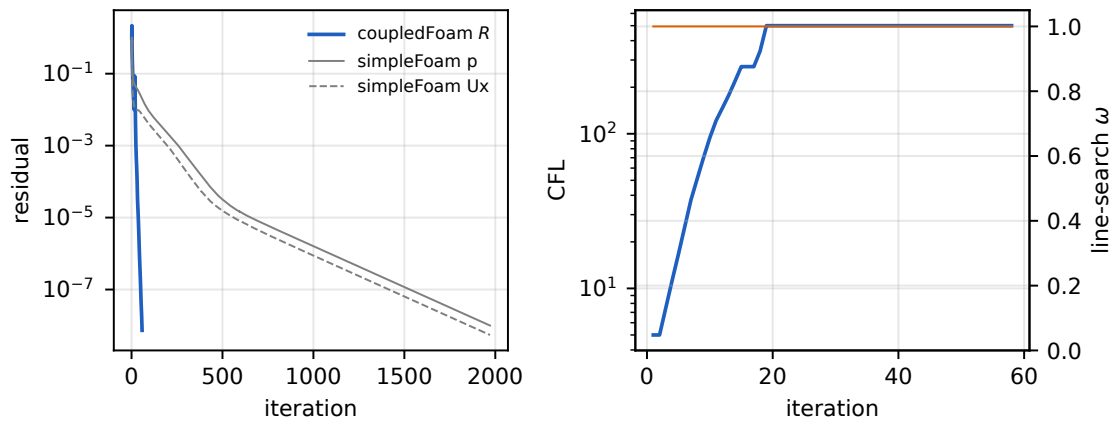


Figure 10: T0, $Re = 1000$, one rank: residuals (left); CFL number and line-search factor ω (right).

in Table 1.

5.4 T1–T5: from 2D turbulence to 3D vehicles

T1, pitzDaily (SST). The OpenFOAM tutorial of a 2D channel with a backward-facing step and a contraction, about 12k cells, turbulent. It tests turbulence coupling and inlet/outlet boundary conditions. Pass: $R < 10^{-6}$ within 400 iterations; pressure drop from inlet to outlet within 1% of `simpleFoam`; the CFL number at least 100 without cuts after iteration 100; no rollbacks. *Why you should care:* the pressure drop is the simplest integral engineering quantity, and the CFL criterion checks that the solver runs at full speed once past the start-up.

T2, backward-facing step (SST). A 2D channel whose floor steps down; the flow separates at the step and reattaches further downstream. Experimental reference: Driver and Seegmiller [9]. Pass: the reattachment length (where the wall shear stress on the lower wall changes sign) within 2% of `simpleFoam`, and at least two times fewer outer iterations to a residual of 10^{-5} (the iteration definition for the two solvers is fixed in D-024). *Why you should care:* separation and reattachment are the phenomena that dominate the wake and diffuser of a car.

T3, airFoil2D (SST and GEKO). The 2D airfoil tutorial. Pass: C_l and C_d within 0.5% of `simpleFoam`, for each turbulence model, with identical settings; empty dynamic remediation set at convergence. *Why you should care:* lift and drag of a wing profile, which is what front and rear wings are.

T4, motorBike. The OpenFOAM motorbike-with-rider tutorial meshed with `snappyHexMesh`: a coarse mesh of about 350k cells (T4a) and a finer mesh (T4b, retargeted to 1–2M cells, D-031), run in parallel. Pass: converged by the force-window criterion; C_d within 1% of `simpleFoam`; static remediation set at most 1% of the cells; no rollbacks. *Why you should care:* this is the first case with a complex 3D geometry and an automatically generated mesh with real bad cells, the closest test to a car.

T5, Ahmed body, 25° slant. A simplified car body with a slanted rear, the standard automotive bluff-body benchmark [1]; half model, 5–8M cells, $U_\infty = 40$ m/s, SST with wall functions. Pass: C_d within 1% of `simpleFoam`; the comparison with the experimental $C_d = 0.285$ ($\pm 10\%$) is for information only. *Why you should care:* it is the size class and flow type of a vehicle simulation: separation on the slant, a wake, ground effect.

Three further tests check specific features:

T-restart. T1 and T3 are split at iteration 150 and restarted: the iteration count must match the continuous run within two iterations, the final quantities within 10^{-5} , and the recomputed fluxes must match the stored ones.

T-fpe. A deliberately brutal start of T1: zero initial velocity, no potential-flow initialisation, start CFL 200, no upwind start-up. The solver must not crash; rollbacks are allowed; it must recover and converge. *Why you should care:* this is the test of the safety net (line search, remediation, sentinel).

T-mrf. The `mixerVessel2D` tutorial with a rotating MRF zone; torque compared with `simpleFoam`. Informational only, because the specification gave no threshold (D-009).

Table 2: Validation, T1–T5: integral quantities (pressure drop, reattachment length, C_l , C_d , torque) of coupledFoam and simpleFoam and their relative differences.

Results pending. Table `tables/validation.tex` is written by `bench/make_report.py` once its data exist.

Results pending. Figure `figures/remediation_history.pdf` is written by `bench/make_report.py` once its data exist.

Figure 11: Remediation set sizes and rollbacks over the iterations for T4 and T5.

How to read Table 2. One row per case and quantity: the value from coupledFoam, the value from simpleFoam on the same mesh, their relative difference, the tolerance of the test, and the verdict. *Good* means: the relative difference is well below the tolerance. A difference close to the tolerance on a case with many remediated cells would point to the local upwinding of Section 4.6.

How to read Figure 11. The number of cells in the dynamic remediation set against the iteration. Typical and healthy: a peak during the early, violent iterations, then decay to (nearly) zero as the flow settles. A set that stays large, or grows late in the run, points at a persistent local problem (mesh or boundary condition).

6 Performance: is it faster?

6.1 The benchmark protocol in plain words

Contestants. For each case (T1, T2, T3 with SST and with GEKO, T4a, T4b, T5), with identical mesh, decomposition, schemes, turbulence model and boundary conditions:

- A** simpleFoam with the tutorial’s own settings (for pitzDaily and the backward-facing step the tutorial already uses SIMPLEC, D-025);
- B** simpleFoam SIMPLEC with relaxation factors 1.0 for p and 0.9 for $\mathbf{U}$, k , ω : the tuned native baseline;
- C** coupledFoam with default settings;
- D** coupledFoam with the simple block-diagonal preconditioner instead of multigrid (shows what the multigrid is worth);
- E** coupledFoam with a V-cycle instead of the K-cycle on T2, T4b and T5 (shows what the cycle choice is worth).

In addition, T1 and T3-SST are run with a fixed linear tolerance instead of Eisenstat–Walker (labelled F in the figures) and with Anderson acceleration (G).

Finish line. Each solver’s own stopping rule is switched off and a fixed number of iterations is run. Afterwards the harness finds, for all solvers in the same way, the first iteration at which the force coefficients have settled: over the last 100 iterations, C_d and C_l vary by at most 0.2% of their mean (for T1 and T2, the pressure drop). This is the “convergence iteration”.

Table 3: Speed-up of coupledFoam against the better native configuration, from the benchmark (values > 1: coupledFoam faster). Generated numbers; “pending” where the benchmark has not been run yet.

case	speed-up (wall-clock)	speed-up (CPU-hours)
T1 pitzDaily	pending	pending
T2 backward-facing step	pending	pending
T3 airfoil, SST	pending	pending
T3 airfoil, GEKO	pending	pending
T4a motorBike, coarse	pending	pending
T4b motorBike, fine	pending	pending
T5 Ahmed body	pending	pending

Table 4: Executive summary: time to convergence (identical window criterion), best native configuration against coupledFoam, as wall-clock time and CPU-hours.

Results pending. Table `tables/executive_summary.tex` is written by `bench/make_report.py` once its data exist.

Stopwatch. Every rank runs under `/usr/bin/time`, which records the wall-clock time and the CPU time. The totals of the whole run are scaled to the convergence iteration using the solver’s own time log (D-025). CPU time is summed over ranks and reported in CPU-hours. Peak memory is the maximum resident memory of each rank. Each configuration is run three times and the median is reported.

Machine. 16 physical cores, 94 GB of memory visible to WSL; MPI ranks bound to physical cores; the heavy cases use at most 10 ranks (D-031); benchmarks run only when no other solver is running, and the machine load before each run is recorded (D-007).

Why it matters: why this protocol is fair

Both solvers are stopped by the same, physically meaningful criterion (the forces do not change any more), not by their incomparable residuals; both get the same hardware and the same mesh; and the native solver gets its best of two settings. A speed-up measured this way is the time you would actually save.

6.2 How to read speed-ups

The speed-up is

$$\text{speed-up} = \frac{\text{time of the better native configuration (A or B)}}{\text{time of coupledFoam (C)}},$$

computed once with wall-clock time and once with CPU-hours. A value above one means coupledFoam is faster; 2.0 means it needs half the time; a value below one means it is slower. The two can differ: if one solver scales worse in parallel, it may still finish sooner in wall time but consume more CPU-hours, or vice versa.

Table 3 extracts the speed-ups; the full executive summary with absolute times and the memory ratio is Table 4.

How to read Table 4. Per case: the wall time and CPU-hours of the best native configuration and of coupledFoam, the two speed-ups, and the memory ratio (coupledFoam’s peak memory

Results pending. Figure `figures/bench_wall.pdf` is written by `bench/make_report.py` once its data exist.

Figure 12: Wall-clock time to convergence (median of three runs) per case and configuration.

Results pending. Figure `figures/bench_cpu.pdf` is written by `bench/make_report.py` once its data exist.

Figure 13: CPU time to convergence (CPU-hours summed over ranks, median of three runs).

divided by the native one; a value above one means coupledFoam needs more memory, which is expected because of the 4×4 blocks).

6.3 The benchmark figures

How to read Figures 12 and 13. One group of bars per case, one bar per configuration (legend). The vertical axis is *logarithmic*: equal distances mean equal *factors*, so a bar one grid step lower is ten times faster. Compare the coupledFoam bar (C) with the lower of the two simpleFoam bars (A, B). Bar D shows the cost of dropping the multigrid, E the cost of the V-cycle.

Figure 14 shows iteration counts. Expect coupledFoam to need far fewer; that alone proves nothing (Section 3.4). The ratio of the iteration ratio to the time ratio tells you how much more expensive a coupled iteration is.

How to read Figure 15. For each case, one bar of coupledFoam’s average time per outer iteration, split into building the system (assembly), solving it (linear solve) and the turbulence update. In a coupled solver the linear solve normally dominates; if assembly or turbulence were dominant, optimising the multigrid would not help much.

Memory (Figure 16). The sum over all ranks of each rank’s peak memory. coupledFoam stores 4×4 blocks and the multigrid hierarchy, so it needs more memory than simpleFoam; the float storage (Section 4.16) halves the matrix part of that overhead. For planning a large case, the ratio between the bars is the number to scale with.

Scaling (Figure 17). *Strong scaling* means: the same mesh, more ranks. Left: time per iteration against the number of ranks, both on log axes; ideal scaling is a straight line falling with slope -1 (twice the ranks, half the time). Right: *parallel efficiency*

$$E(n) = \frac{t(n_0) n_0}{t(n) n},$$

Results pending. Figure `figures/bench_iters.pdf` is written by `bench/make_report.py` once its data exist.

Figure 14: Outer iterations to convergence. Shown for completeness; the decisive metrics are those of Figures 12 and 13.

Results pending. Figure `figures/bench_breakdown.pdf` is written by `bench/make_report.py` once its data exist.

Figure 15: coupledFoam time per outer iteration split into assembly, linear solve and turbulence.

Results pending. Figure `figures/bench_memory.pdf` is written by `bench/make_report.py` once its data exist.

Figure 16: Peak memory, sum of the per-rank maximum RSS.

with $t(n)$ the time per iteration on n ranks and n_0 the smallest rank count; $E = 1$ is perfect, $E = 0.5$ means half of the added cores are wasted on communication and waiting. On this machine the scaling test runs on 1, 2, 4, 8 and 10 ranks (D-031; the specification originally planned 16), with the pass threshold unchanged. Multigrid solvers typically lose some efficiency on coarse levels; processor agglomeration (Section 4.15) is the countermeasure.

How to read Table 5. For each run: how many multigrid levels were built, the `mergeLevels` value finally used (larger than 2 means the coarsening rule forced more aggressive coarsening), the operator complexity C_{op} (total size of all level matrices relative to the finest; must be at most 1.5), and the number of cells on each level. Dividing consecutive cell counts gives the coarsening ratios of Section 4.11.

Cycles (Figure 18). Krylov iterations (left) and solve time (right) per cycle type. A stronger cycle needs fewer Krylov iterations but each is more expensive; the time panel decides.

Controller histories (Figure 19). Left: the linear tolerance η_n chosen by Eisenstat–Walker, on a log axis. Expect it at 0.5 during the start-up and falling as the outer iteration speeds up. Right: the measured cycle efficiency ρ (lower is better); dotted vertical lines mark changes made by the autoTune controller.

Anderson (Figure 20). Pairs of bars, off and on, for iterations (left) and wall time (right). Anderson is worth its memory only if the right panel shows a clear gain.

7 What this means for the F1 half-car

The project’s readiness check (`F1_READINESS.md`) went through the F1 half-car case (the most recent stage, run with `simpleFoam` and k - ω SST) feature by feature. In plain language:

Results pending. Figure `figures/scaling.pdf` is written by `bench/make_report.py` once its data exist.

Figure 17: Strong scaling on T4b: time per iteration and parallel efficiency of coupledFoam and simpleFoam. Pass: the parallel efficiency of coupledFoam at the largest rank count is at least 80 % of that of simpleFoam.

Table 5: Block-GAMG hierarchy per run: cycle type and post-smoothing sweeps at start and end (autoTune), levels, operator complexity, cells, ranks and coarsening ratios per level, coarsest-level solver. Ranks per level are n/a for logs written before D-030.

run	cycle (start/end)	post-sweeps	levels	mergeLevels	C_op	cells per level	ranks per level	coarsening ratios	coarsest solver
T0_Re1000_np1	K/K	2/2	4	2	1.325	16384 4096 1024 256	n/a	4 4 4	blockBiCGStab
T0_Re100_np1	K/K	2/2	4	2	1.325	16384 4096 1024 256	n/a	4 4 4	blockBiCGStab

Results pending. Figure `figures/cycle_comparison.pdf` is written by `bench/make_report.py` once its data exist.

Figure 18: Cycle types: wall time and Krylov iterations with the V-cycle (configuration E) against the default K-cycle.

feature of the F1 case	status in coupledFoam
inlet (fixed velocity), outlet (inletOutlet velocity, fixed pressure)	supported through the generic boundary treatment (Section 4.1)
slip tunnel walls	supported; on the review list to double-check
no-slip car and driver, moving road, rotating wheel walls	supported
symmetry plane, processor patches	supported (symmetry: zero flux; processor: fully implicit)
MRF zones for the wheels, rear zone with nonRotatingPatches	MRF implicit (Section 4.18); nonRotatingPatches to be verified before the first F1 run
k- ω SST	supported, segregated after each coupled step
bounded Gauss linearUpwind limited for div(phi,U)	supported through deferred correction (Section 4.2)
open domain with fixed-pressure outlet	good: no pressure reference needed (Section 4.17)
no cyclicAMI, no mesh changes during the run	fine; coupledFoam does not support topology changes at run time
force function objects	work; solverInfo shows empty entries for the block solve (the coupledFoam log line and summary file carry that information)

Switching the case over. The readiness file contains a drop-in `fvSolution` block. What its main entries mean, in terms of this document:

- `solver blockFGMRES; preconditioner blockGAMG;`: the Krylov solver and multigrid of Sections 4.9–4.11.
- `adaptiveRelTol yes;`: Eisenstat–Walker (Section 4.13).
- `cycleType K;` with the comment “use V until the K-cycle fix lands”: see the open items below.
- `smoother blockILU0;`: the smoother of the learning story (Section 4.12).
- `autoTune yes;`: the cycle-efficiency controller.

Results pending. Figure `figures/eta_rho_history.pdf` is written by `bench/make_report.py` once its data exist.

Figure 19: Eisenstat–Walker forcing term η_n and preconditioner efficiency ρ with the controller events.

Results pending. Figure `figures/anderson.pdf` is written by `bench/make_report.py` once its data exist.

Figure 20: Anderson acceleration on and off (T1, T3-SST): outer iterations and wall time.

- The coupled dictionary with `potentialInit yes`: start from a `potentialFoam` solution; the `convergence` entries set a residual target and the force-window criterion (100 iterations, 0.2%). By default the run stops when *either* criterion is met (D-005).
- The turbulence solvers for k and ω stay as they are.

What to watch in the log. `coupledFoam` prints one line per outer iteration starting with `CFL`, containing the CFL number, the line-search factor ω , the number of CFL cuts, R , r_U , r_p , the Eisenstat–Walker tolerance, linear iterations and residual, the times for assembly, solve and turbulence, the sizes of the remediation sets, the rollback count and the number of clamped values. For a first F1 run, the useful questions are: does the CFL number climb and stay high (Section 4.4)? Is ω mostly one (Section 4.5)? Do the remediation sets stay small (Section 4.6)? Do the linear solves reach their tolerance (Section 4.12)? And, finally, do the force coefficients settle? At the end, a JSON summary with iterations, wall and CPU time and peak memory is written.

Open items before a production F1 run. The readiness check lists, at the time of writing: the verification of the `slip` linearisation and of MRF `nonRotatingPatches`; an open issue in parallel runs with the K-cycle that is being investigated (hence the comment to use the V-cycle until it is fixed); the formal smoother decision on T2; and the validation on T3 and T4. The results of T3–T5 and of the benchmark in this document (Sections 5.4 and 6) are exactly the evidence those items require.

Why it matters: what to expect on the F1 case

The benchmark cases T4 and T5 are the closest to the F1 half-car: complex 3D geometry, `snappyHexMesh` meshes with bad cells, separated wakes, and parallel runs. Their rows in Tables 3 and 4 and the memory figure are the best available predictor of what `coupledFoam` will do on the F1 case. The F1 case adds rotating MRF wheels and a moving ground, which T-mrf covers only informally.

8 Limitations

- **Memory.** A coupled solver needs more memory than a segregated one. The budget for the largest target mesh (45 M cells) cannot be verified on the development machine (94 GB); the largest test meshes are T4b and T5.
- **Periodic and AMI patches** are coupled explicitly (lagged values), which slows convergence across them; implicit coupling is planned for Phase 2.
- **The pseudo-time step enters the Rhie–Chow term**, so the converged solution depends very slightly on the final time step, as a SIMPLE solution depends on its relaxation factor (Section 4.3).
- **Gradient limiting in bad cells** applies to the Rhie–Chow pressure gradient only; the gradient inside your convection scheme cannot be limited per cell (D-018). With the default settings, bad cells use pure upwind, so this gradient does not enter there.

- **Some boundary conditions are linearised about the previous iteration** (`inletOutlet`, `totalPressure`); outflow through a fixed-value velocity face uses the prescribed value (D-019).
- **MRF** is implemented implicitly but checked only by an informational test (D-009).
- **The restart file is written atomically** only with the default (uncollated) file handler (D-017).
- **Residuals** of `coupledFoam` and `simpleFoam` are not directly comparable; all speed comparisons use the force window.

Phase 2 (not in this version): a block-selective algebraic multigrid [28, 29] as an additional preconditioner, adaptive mesh refinement during the run (the restart format already reserves space for it), implicit cyclic and AMI coupling, GPU execution, and the 45 M-cell production run with a measured memory budget.

9 Summary: the ideas in ten sentences

1. Pressure has no equation of its own; it is whatever makes the flow conserve mass.
2. `simpleFoam` handles this by letting velocity and pressure take turns, with under-relaxation, which is cheap per iteration but needs many iterations.
3. `coupledFoam` solves velocity and pressure together in one system of 4×4 blocks, which needs far fewer, but more expensive, iterations.
4. Only wall-clock time and CPU-hours to the same converged forces decide which is faster.
5. The Rhie–Chow interpolation prevents checkerboard pressure and makes the coupled system solvable.
6. A pseudo-time step per cell, steered by an adaptive CFL number, replaces under-relaxation and does not change the converged answer.
7. Line search, problem-cell sets and rollback keep single bad steps or bad cells from ruining a run.
8. The linear system is solved by FGMRES with a block algebraic multigrid, to a tolerance that follows the progress of the outer iteration.
9. The quality of the smoother decides whether the outer iteration runs smoothly: block ILU(0) succeeded where block Gauss–Seidel failed.
10. The big data are stored in float for speed, while every residual and sum is computed in double, so the final accuracy is that of double precision.

Glossary

Agglomeration. Gluing neighbouring cells into coarse cells to build the coarse levels of algebraic multigrid (Section 4.11).

Algebraic multigrid (AMG, GAMG). A solver or preconditioner that reduces errors of all sizes by working on a hierarchy of coarsened versions of the matrix.

Anderson acceleration. Extrapolating the next outer update from a short history of previous updates; optional in coupledFoam.

Assembly. Computing all matrix entries and the right-hand side from the current fields.

BiCGStab. A Krylov method with low, fixed memory; its residual does not decrease monotonically.

Block. The 4×4 sub-matrix that couples the four unknowns (u, v, w, p) of one cell with those of itself or a neighbour.

Boundary linearisation. Writing a boundary value as factor $\times$ cell value + constant, which OpenFOAM provides for every patch type.

C_d, C_l . Drag and lift coefficients: forces divided by dynamic pressure times reference area.

CFL number. Roughly, how many cells a fluid parcel crosses in one (pseudo-)time step; controls the size of the pseudo-time step.

Checkerboard mode. An alternating pressure pattern that a naive cell-centred discretisation cannot see; removed by Rhie–Chow.

Continuity. Conservation of mass; for incompressible flow $\nabla \cdot \mathbf{U} = 0$.

Convection. Transport of a quantity (here momentum) by the flow itself.

Coupled solver. A solver that solves velocity and pressure simultaneously in one linear system.

CPU-hours. CPU time added up over all ranks; the computing cost of a run.

Cycle (V, F, W, K). The order in which multigrid visits its levels.

Deferred correction. Keeping a robust low-order scheme in the matrix and adding the difference to the high-order scheme as a known source.

Double precision. 8-byte floating-point numbers, about 16 digits.

Eisenstat–Walker. A rule that sets the inner (linear) tolerance from the progress of the outer iteration.

FGMRES. Flexible GMRES; tolerates a preconditioner that changes from one application to the next.

Finite volume. Discretisation by balancing fluxes over the faces of each cell.

Float (single precision). 4-byte floating-point numbers, about 7 digits.

Flux (ϕ). Volume per second crossing a face.

GMRES. A Krylov method that minimises the residual over all directions generated so far; restarted to limit memory.

Increment form. Solving for the change of the fields, with the residual as right-hand side; makes the float solve safe.

Inner iteration. One iteration of the linear solver inside an outer iteration.

Krylov method. An iterative linear solver built from repeated matrix–vector products.

Latency. The waiting time for a message between ranks, independent of its size.

Line search. Shortening a proposed update by a factor $\omega \leq 1$ so that no cell changes too much.

Local time step. A pseudo-time step chosen separately for each cell.

LDU matrix. OpenFOAM’s sparse matrix format: diagonal, and upper and lower entries per face.

Memory bandwidth. How fast data can be moved from memory to the processor; the usual bottleneck of sparse linear algebra.

MRF. Multiple reference frame: a zone computed in a rotating frame to represent rotating parts (e.g. wheels) in a steady run.

Operator complexity. Total size of all multigrid level matrices relative to the finest matrix.

Outer iteration. One cycle of assemble, solve, update, turbulence; the iteration counted in the log.

Parallel efficiency. How well added cores are used: time on n_0 ranks times n_0 , divided by time on n ranks times n .

Preconditioner. A cheap approximate inverse that makes a Krylov method converge in far fewer iterations.

Processor agglomeration. Gathering coarse multigrid levels onto fewer ranks to save communication.

Pseudo-transient continuation (PTC). Marching a steady problem with fake time steps chosen for robustness and speed.

RANS. Reynolds-averaged Navier–Stokes: equations for the time-averaged turbulent flow, closed by a turbulence model.

Rank. One MPI process of a parallel run, owning one piece of the mesh.

Remediation. Local, gentler numerics (upwind, smaller time step) in cells with bad mesh quality or bad behaviour.

Residual. How strongly the current fields violate the discrete equations; zero at the discrete solution.

Rhie–Chow interpolation. A correction of the face flux that couples neighbouring pressures and removes the checkerboard mode.

Rollback. Restoring the fields of the previous iteration after a failed sentinel check.

Saddle-point system. A system with a zero diagonal block, as the coupled system would be without Rhie–Chow.

Segregated solver. A solver that solves velocity and pressure one after the other (SIMPLE, PISO).

Sentinel. The check after each update for non-finite or absurdly large values.

SIMPLE, SIMPLEC. The classical segregated algorithms for steady incompressible flow used by `simpleFoam`.

Smoother. A cheap local iteration (Gauss–Seidel, ILU(0)) that removes jagged errors on each multigrid level.

Speed-up. Native time divided by coupledFoam time; above one means coupledFoam is faster.

Strong scaling. Same mesh, more ranks: how the time per iteration falls.

Under-relaxation. Accepting only a fraction of each change to keep a segregated iteration stable.

Upwind. Taking a face value from the upstream cell; robust but diffusive.

Wall-clock time. Elapsed time from start to finish of a run.

References

- [1] S. R. Ahmed, G. Ramm, and G. Faltin. Some salient features of the time-averaged ground vehicle wake. Technical Report 840300, SAE, 1984.
- [2] D. G. Anderson. Iterative procedures for nonlinear integral equations. *Journal of the ACM*, 12(4):547–560, 1965.
- [3] M. Baboulin, A. Buttari, J. Dongarra, J. Kurzak, J. Langou, J. Langou, P. Luszczek, and S. Tomov. Accelerating scientific computations with mixed precision algorithms. *Computer Physics Communications*, 180(12):2526–2533, 2009.
- [4] E. Carson and N. J. Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817–A847, 2018.
- [5] M. Ceze and K. J. Fidkowski. A robust adaptive solution strategy for high-order implicit CFD solvers. In *20th AIAA Computational Fluid Dynamics Conference*, 2011. AIAA Paper 2011-3696.
- [6] M. Ceze and K. J. Fidkowski. Pseudo-transient continuation, solution update methods, and CFL strategies for DG discretizations of the RANS-SA equations. In *21st AIAA Computational Fluid Dynamics Conference*, 2013. AIAA Paper 2013-2686.
- [7] M. Darwish, I. Sraj, and F. Moukalled. A coupled incompressible flow solver on structured grids. *Numerical Heat Transfer, Part B: Fundamentals*, 52(4):353–371, 2007.
- [8] M. Darwish, I. Sraj, and F. Moukalled. A coupled finite volume solver for the solution of incompressible flows on unstructured grids. *Journal of Computational Physics*, 228(1):180–201, 2009.
- [9] D. M. Driver and H. L. Seegmiller. Features of a reattaching turbulent shear layer in divergent channel flow. *AIAA Journal*, 23(2):163–171, 1985.
- [10] S. C. Eisenstat and H. F. Walker. Choosing the forcing terms in an inexact Newton method. *SIAM Journal on Scientific Computing*, 17(1):16–32, 1996.
- [11] J. H. Ferziger and M. Perić. *Computational Methods for Fluid Dynamics*. Springer, Berlin, 3rd edition, 2002.
- [12] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982.
- [13] B. R. Hutchinson and G. D. Raithby. A multigrid method based on the additive correction strategy. *Numerical Heat Transfer*, 9(5):511–537, 1986.
- [14] C. T. Kelley and D. E. Keyes. Convergence analysis of pseudo-transient continuation. *SIAM Journal on Numerical Analysis*, 35(2):508–523, 1998.
- [15] P. K. Khosla and S. G. Rubin. A diagonally dominant second-order accurate implicit scheme. *Computers & Fluids*, 2(2):207–209, 1974.
- [16] D. A. Knoll and D. E. Keyes. Jacobian-free Newton–Krylov methods: a survey of approaches and applications. *Journal of Computational Physics*, 193(2):357–397, 2004.

- [17] L. Mangani, M. Buchmayr, and M. Darwish. Development of a novel fully coupled solver in OpenFOAM: steady-state incompressible turbulent flows. *Numerical Heat Transfer, Part B: Fundamentals*, 66(1):1–20, 2014.
- [18] L. Mangani, M. Buchmayr, and M. Darwish. Development of a novel fully coupled solver in OpenFOAM: steady-state incompressible turbulent flows in rotational reference frames. *Numerical Heat Transfer, Part B: Fundamentals*, 66(6):526–543, 2014.
- [19] F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.
- [20] F. R. Menter and A. Matyushenko. Generalized k - ω (GEKO) two-equation turbulence model. *AIAA Journal*, 63(11):4590–4606, 2025. doi: 10.2514/1.J065393.
- [21] W. A. Mulder and B. Van Leer. Experiments with implicit upwind methods for the Euler equations. *Journal of Computational Physics*, 59(2):232–246, 1985.
- [22] Y. Notay. An aggregation-based algebraic multigrid method. *Electronic Transactions on Numerical Analysis*, 37:123–146, 2010.
- [23] Y. Notay and P. S. Vassilevski. Recursive Krylov-based multigrid cycles. *Numerical Linear Algebra with Applications*, 15(5):473–487, 2008.
- [24] S. V. Patankar and D. B. Spalding. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer*, 15(10):1787–1806, 1972.
- [25] C. M. Rhie and W. L. Chow. Numerical study of the turbulent flow past an airfoil with trailing edge separation. *AIAA Journal*, 21(11):1525–1532, 1983.
- [26] Y. Saad. A flexible inner-outer preconditioned GMRES algorithm. *SIAM Journal on Scientific Computing*, 14(2):461–469, 1993.
- [27] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2nd edition, 2003.
- [28] T. Uroić and H. Jasak. Block-selective algebraic multigrid for implicitly coupled pressure-velocity system. *Computers & Fluids*, 167:100–110, 2018.
- [29] T. Uroić and H. Jasak. Parallelisation of selective algebraic multigrid for block-pressure-velocity system in OpenFOAM. *Computer Physics Communications*, 258:107529, 2021.
- [30] J. P. Van Doormaal and G. D. Raithby. Enhancements of the SIMPLE method for predicting incompressible fluid flows. *Numerical Heat Transfer*, 7(2):147–163, 1984.
- [31] S. P. Vanka. Block-implicit multigrid solution of Navier–Stokes equations in primitive variables. *Journal of Computational Physics*, 65(1):138–158, 1986.
- [32] H. F. Walker and P. Ni. Anderson acceleration for fixed-point iterations. *SIAM Journal on Numerical Analysis*, 49(4):1715–1735, 2011.
- [33] J. M. Weiss, J. P. Maruszewski, and W. A. Smith. Implicit solution of preconditioned Navier–Stokes equations using algebraic multigrid. *AIAA Journal*, 37(1):29–36, 1999.